Lab 11 — Questions

Question 1 [Understanding]

A colleague claims: "With `restart: always` and two Compose VMs behind a load balancer, I have the same thing as Kubernetes." Name three concrete situations where that architecture fails and a cluster copes, and for each name the control plane component that acts.

Question 2 [Prediction]

```
kubectl apply -f pod-api.yaml
kubectl apply -f pod-api.yaml
kubectl create -f pod-api.yaml
```

Predict the output of each of the three commands, then explain what "declarative" and "idempotent" mean through this example.

Question 3 [Prediction]

```
kubectl run web --image=docker.io/library/nginx:alpine
kubectl create deployment api --image=localhost/api:1.0
kubectl delete pod web
kubectl delete pod $(kubectl get pods -l app=api -o jsonpath='{.items[0].metadata.name}')
kubectl get pods
```

What does the last `get pods` show, and why do the two deletions have different effects? Which object, absent in the first case, makes the difference?

Question 4 [Diagnosis]

```
$ podman images | grep api
localhost/api   1.0   dd522f8992fd   2 days ago   209 MB
$ kubectl apply -f pod-api.yaml && sleep 10 && kubectl get pods
NAME   READY   STATUS             RESTARTS   AGE
api    0/1     ImagePullBackOff   0          10s
```

The image does exist. Explain what the kubelet tried, why it cannot find the image, and give the command that fixes the problem as well as the manifest field that matters.

Question 5 [Diagnosis]

`kubectl get pods` answers `No resources found in default namespace.` while `kubectl get pods -A` does list your `api` Pod. Explain, give two ways to see the Pod, and the command that avoids repeating `-n` every time.

Question 6 [Analysis]

For the `web → api` chain, two designs: (a) a single Pod with two containers `web` and `api`; (b) two Pods, one per service. How does nginx refer to the API in each case? Which one resembles lab 07, which one lab 09? And which one is chosen in production, and why?

Question 7 [Analysis]

Open `/etc/rancher/k3s/k3s.yaml`: what does it contain, in three elements? Why is `--write-kubeconfig-mode 644` acceptable on your workstation and refused on a shared server? And what happens if you copy that file as is to another machine to drive your cluster?

Question 8 [Diagnosis]

```
$ kubectl get pods
The connection to the server localhost:8080 was refused - did you specify the right host or port?
```

Yet `sudo systemctl status k3s` says `active (running)`. What is `kubectl` looking for, why does it target `localhost:8080`, and what are the two possible fixes?

Question 9 [Understanding]

If the API server stops for ten minutes, what becomes of the Pods running on the nodes? What can you no longer do? And what does the kubelet do with a Pod that dies during that outage? Deduce why the API server is said to be "the only door", not "the engine".

Question 10 [Analysis]

Here are the Events of a `kubectl describe pod web`:

```
Normal  Scheduled  default-scheduler  Successfully assigned lab11/web to wsl-ubuntu
Normal  Pulling    kubelet            Pulling image "docker.io/library/nginx:alpine"
Normal  Pulled     kubelet            Successfully pulled image … in 3.75s
Normal  Created    kubelet            Created container: web
Normal  Started    kubelet            Started container web
```

Attribute each line to a component and say what it did; then set them against the steps of a `podman run` from lab 03. What does the cluster do in addition, and who does nothing here because there is only one node?

Question 11 [Diagnosis]

A Pod has been `Pending` for five minutes, with no IP at all in `kubectl get pods -o wide`. What are the two most likely causes, which command tells them apart, and what does the message look like in each case?

Question 12 [Analysis]

A developer who has only known k3s joins a team on EKS. Name four things that work "by themselves" on k3s and that they will have to learn to request or configure on a managed cluster, and two things that do not change at all.

Question 13 [Analysis]

The YAML produced by `podman generate kube` in the hands-on lab contains a `hostPath` volume pointing to `~/labo-docker/07/nginx-pod.conf` and a `hostPort: 18090`. Explain why those two fields are acceptable on your machine and unacceptable in a cluster manifest, and which Kubernetes objects replace them.